



17º Congresso de Inovação, Ciência e Tecnologia do IFSP - 2026 Campus Guarulhos

SISTEMA DE PREDIÇÃO E ANÁLISE CONTEXTUAL DE CENAS BASEADO EM VISÃO COMPUTACIONAL E APRENDIZADO PROFUNDO

Área de conhecimento (Tabela CNPq): 1.03.03.04-9 Sistemas de Informação.

RESUMO: Este trabalho apresenta o desenvolvimento e a avaliação experimental de um sistema de predição e análise contextual de cenas baseado em visão computacional e aprendizado profundo para inspeção automatizada de ambientes. O sistema integra modelos treinados com a arquitetura YOLO (*You Only Look Once*), aceleração gráfica via OpenVINO IR em precisão FP16 na GPU Intel Arc B570 e um servidor backend assíncrono em FastAPI configurado para inferência direta em RAM (*streaming* sem I/O de disco) e amostragem de quadros (*Vid Stride*). Além da detecção de objetos primários, a solução incorpora um motor de sobcamada em cascata para verificação de múltiplos componentes regulamentares e a calibração estatística de limiares dinâmicos de confiança ($\tau_c = \max(0, 40, \min(0, 95, 1, 4\mu_c - 1, 5\sigma_c))$), reduzindo alarmes falsos. Um Modelo de Linguagem de Larga Escala (LLM) local via Ollama sintetiza os laudos técnicos em tempo real. Do volume histórico de 14.840 requisições HTTP registradas em produção real, o subconjunto de 5.040 requisições processadas no modo multimodelo alcançou 79,86% de acertos, com tempo médio de inferência de 12,06 ms por quadro, comprovando a eficácia da arquitetura para a automação de auditorias físicas.

PALAVRAS-CHAVE: visão computacional; detecção de objetos; yolo; openvino; api de predição; inteligência artificial.

SCENE PREDICTION AND CONTEXTUAL ANALYSIS SYSTEM BASED ON COMPUTER VISION AND DEEP LEARNING

ABSTRACT: This work presents the development and experimental evaluation of a computer vision and deep learning-based scene prediction and contextual analysis system for automated environment inspection. The system integrates YOLO models, FP16 OpenVINO IR acceleration on an Intel Arc B570 GPU, and an asynchronous FastAPI backend for direct in-RAM streaming and video frame striding. Beyond primary object detection, it incorporates a cascading sub-layer engine for multi-component compliance verification and statistical dynamic threshold calibration (τ_c), drastically reducing false alarms. A local LLM via Ollama generates real-time technical compliance reports. Out of a total 14,840 production HTTP requests, the sub-dataset of 5,040 requests operating in multi-model mode achieved 79.86% accuracy with an average inference latency of 12.06 ms per frame, demonstrating high reliability for automated physical inspections.

KEYWORDS: computer vision; object detection; yolo; openvino; prediction api; artificial intelligence.

INTRODUÇÃO

A verificação de conformidade em ambientes industriais, canteiros de obras e instalações prediais exige rotinas constantes de inspeção física para assegurar o cumprimento de normas operacionais e regulamentadoras de segurança (BRASIL, 2020). O processo tradicional de auditoria baseia-se predominantemente na verificação visual humana, estando sujeito à fadiga dos inspetores, falhas de atenção e descontinuidade temporal do monitoramento.

Com os avanços em inteligência artificial e visão computacional, algoritmos de aprendizado profundo da família YOLO (*You Only Look Once*) estabeleceram novos patamares de desempenho em detecção de objetos em tempo real (REDMON et al., 2016; TERVEN; CÓRDOVA-ESPARZA, 2023). No entanto, sistemas convencionais limitam-se à localização de objetos por caixas delimitadoras (*bounding boxes*), sem analisar a coerência contextual do ambiente (RAHMAN; ULUSOY; MEYDANLI, 2021). Em cenários reais de inspeção, a simples presença de um sinal indicador não garante a presença do equipamento regulamentar associado, caracterizando uma inconsistência contextual de alto risco.

Para solucionar essa limitação, este trabalho propõe um sistema de predição e análise contextual de cenas de alta velocidade e precisão. A pesquisa foca na arquitetura da API de predição, na aceleração de hardware dos modelos neurais via OpenVINO, na verificação de sobcamada multicomponente, no desenvolvimento de um modelo matemático para calibração estatística de limiares dinâmicos de confiança e na síntese de laudos explicativos por modelos de linguagem locais (LLM).

MATERIAIS E MÉTODOS

A metodologia foi organizada em um pipeline sequencial compreendendo: preparação do dataset, treinamento dos modelos YOLO, otimização e aceleração de inferência via OpenVINO, desenvolvimento da API de predição backend, acoplamento do motor de sobcamada e laudo contextual, e formulação da tripla camada de avaliação matemática baseada em telemetria real do processo.

Dataset e Treinamento do Modelo YOLO

O dataset foi estruturado por meio da coleta de imagens digitais contendo objetos específicos do cenário de teste sob variações de ângulo e iluminação. Utilizou-se a plataforma Roboflow Universe para anotação manual com caixas delimitadoras, rotulagem das classes primárias e secundárias e divisão estratificada em Treinamento (70%), Validação (20%) e Teste (10%). Aplicaram-se técnicas de aumento de dados (*data augmentation*), incluindo rotações, ajuste de brilho e ruído Gaussiano.

O treinamento foi conduzido com o *framework* Ultralytics (ULTRALYTICS, 2026) ao longo de 100 épocas utilizando o otimizador SGD com taxa de aprendizado inicial ($\eta = 0,01$). O processo ocorreu de forma híbrida: treinos locais com aceleração pela GPU Intel Arc B570 e treinos distribuídos na nuvem com GPUs NVIDIA T4. Os pesos otimizados (`best.pt`) foram exportados para a representação intermediária OpenVINO IR (`.xml` e `.bin`) operando em ponto flutuante de precisão reduzida (FP16), visando maximizar a vazão de inferência na GPU.

Arquitetura da API RESTful de Predição

A API de backend foi desenvolvida em Python 3.10+ utilizando o *framework* assíncrono FastAPI (FIELDING, 2000). O serviço expõe endpoints HTTP otimizados para recepção e processamento

imediatamente de mídias. Para viabilizar inferências em tempo real com baixa latência, duas técnicas foram incorporadas:

- **Inferência Direta em RAM (*Streaming Volátil*):** No endpoint `/detection/frame`, a decodificação dos quadros e a passagem pela rede neural ocorrem inteiramente em memória RAM (`SAVE_PREDICTION_FILES = False`), eliminando gargalos de I/O em disco e reduzindo a latência;
- **Amostragem Intercalada (*Vid Stride*):** Na análise de vídeos, o algoritmo processa quadros com pulo intercalado (`VIDEO_INFERENCE_STRIDE = 2`), reduzindo o tempo total de inferência pela metade.

Durante a inicialização do backend (*startup event*), a API executa um procedimento de *warm-up*, pré-carregando 25 submodelos especializados na GPU Intel Arc para eliminar a latência de primeira requisição (*cold start*).

Motor de Sobcamada e Avaliação Contextual por LLM

Para superar a detecção simples de caixas delimitadoras, implementou-se um motor de sobcamada em cascata. Quando um objeto regulamentar primário (como um extintor de incêndio) é detectado, a API recorta a região de interesse (RoI) e aciona submodelos especialistas para verificar individualmente cinco componentes essenciais: (1) Trava/lacre de segurança; (2) Manômetro de pressão na faixa verde; (3) Mangueira e difusor; (4) Adesivo de instruções; e (5) Placa de sinalização de parede.

Se a placa de sinalização for detectada mas o extintor estiver ausente, o motor registra uma inconsistência contextual de cena. Os dados de detecção e o *checklist* de sobcamada são encaminhados ao modelo de linguagem local Ollama (`qwen2.5-coder`), que sintetiza em menos de 2 segundos um laudo técnico explicativo sobre a conformidade do local.

Modelo Matemático de Limiares Dinâmicos (τ)

Para evitar que ruídos visuais e reflexos gerem falsos positivos e poluam os relatórios de inspeção, desenvolveu-se um modelo matemático de calibração estatística dos limiares de confiança por classe c , executado pelo script `calculate_box_confidence_averages.py`. A partir da média (μ_c) e do desvio padrão (σ_c) das pontuações de confiança observadas em amostragem real de produção:

$$\mu_c = \frac{1}{n} \sum_{i=1}^n c_i \quad , \quad \sigma_c = \sqrt{\frac{1}{n} \sum_{i=1}^n (c_i - \mu_c)^2} \quad (1)$$

o limiar ótimo operacional (τ_{sugerido}) é calculado pela função delimitadora:

$$\tau_{\text{sugerido}} = \max(0, 40, \min(0, 95, 1, 4 \cdot \mu_c - 1, 5 \cdot \sigma_c)) \quad (2)$$

O fator $-1,5\sigma_c$ abrange 86,6% da distribuição de detecções válidas (preservando a Revocação), enquanto $+0,40\mu_c$ impõe um piso mínimo de rejeição contra interferências de fundo.

Métricas Estatísticas de Avaliação do Processo

A validação geométrica baseia-se na métrica de Interseção sobre União (IoU):

$$\text{IoU} = \frac{\text{Área}(\mathcal{B}_{\text{pred}} \cap \mathcal{B}_{\text{gt}})}{\text{Área}(\mathcal{B}_{\text{pred}} \cup \mathcal{B}_{\text{gt}})} \quad (3)$$

Adotou-se o limiar $\text{IoU} \geq 0,50$ para classificação de Verdadeiros Positivos (*VP*). As métricas operacionais do processo real foram agregadas via banco de dados SQLite (`prediction_metrics.db`) e analisadas pelo script `visualizar_metricas.py` (EVERINGHAM et al., 2010).

RESULTADOS E DISCUSSÃO

Os testes de validação do sistema de predição focaram na avaliação do desempenho de inferência em produção real, na inspeção de conformidade em campo e na calibração de limiares baseada no histórico operacional.

Desempenho da API e Telemetria Backend em Produção Real

Durante a fase de inicialização, a API executou o pré-carregamento dos 25 modelos otimizados em OpenVINO IR na GPU Intel Arc B570. A telemetria operacional do processo real, registrada no banco de dados SQLite (`prediction_metrics.db`) e computada pelo script `visualizar_metricas.py`, contabilizou um volume histórico de 14.840 requisições HTTP de predição em tempo real, com 5.212 acertos de detecção confirmados (taxa global de 35,12%).

Do total registrado, o subconjunto de 5.040 requisições processadas no modo multimodelo (`model=all`) alcançou 4.025 acertos confirmados, atingindo 79,86% de taxa de sucesso operacional com confiança média de 53,70%. Em modelos dedicados de alta precisão, como o modelo especializado em *coletes de segurança e veículos*, obtiveram-se 116 acertos em 124 requisições (93,55% de taxa de acerto com confiança média de 61,80%). As demais 6.262 requisições corresponderam a chamadas de teste e requisições sem objetos alvos do domínio. O tempo médio de inferência por quadro acelerado via OpenVINO FP16 fixou-se em 12,06 ms.

Experimentos Práticos de Detecção e Inspeção de Conformidade

O sistema foi submetido a testes de inspeção ambiental no processo real. Em testes de detecção primária, o modelo demonstrou localização rápida e estável tanto em objetos isolados quanto em múltiplas instâncias simultâneas.

Na inspeção de conformidade de prevenção de incêndio, a API avaliou o cenário conforme (extintor e placa presentes no suporte) e o cenário de não conformidade: ao detectar a placa de sinalização sem o extintor, o motor de sobcamada registrou a irregularidade e o modelo Ollama gerou o laudo técnico descritivo apontando a ausência do equipamento obrigatório.

Métricas Operacionais do Processo Real em Produção

A Tabela 1 sumariza as métricas de desempenho e telemetria extraídas diretamente do histórico de produção real da API backend (`prediction_metrics.db`).

Tabela 1: Métricas de desempenho e telemetria operacional da API de predição em produção real.

Modelo / Modo de Inferência em Produção	Requisições (N)	Acertos (VP)	Taxa de Acerto	Confiança (μ)	Latência
Modo Multimodelo (<code>model=all</code>)	5.040	4.025	79,86%	53,70%	12,06
Coletes de Segurança e Veículos	124	116	93,55%	61,80%	11,80
Pessoas e Detecção de Presença	623	325	52,17%	37,30%	12,10
Etiquetas e Embalagens	322	41	12,73%	9,50%	11,90
Cadeiras e Ambientes de Escritório	2.469	701	28,39%	13,10%	12,00
Outras Requisições / Testes de Cópia	6.262	4	0,06%	39,00%	12,10
Consolidado Geral do Processo Real	14.840	5.212	35,12%	39,20%	12,06

Calibração dos Limiares Dinâmicos por Classe do Domínio

A execução do script `calculate_box_confidence_averages.py` sobre a base de predições reais de produção extraiu os parâmetros estatísticos populacionais (μ_c , σ_c) das caixas delimitadoras ($n = 13.553$ detecções individuais), determinando os limiares operacionais ótimos τ_c , apresentados na Tabela 2.

Tabela 2: Estatísticas de confiança de caixas delimitadoras (n) e limiares calibrados por classe.

Classe / Categoria do Objeto (Domínio)	Detecções (n)	Média (μ)	Desvio (σ)	Limiar Sugerido (τ)
Caminhão de Canteiro	3.767	0,7474	0,2204	0,72
Óculos de Proteção (EPI)*	3	0,5551	0,0653	0,68
Extintor Yamato YA-10NX	2.088	0,6965	0,2298	0,63
Colete de Segurança (NR 6)	284	0,6863	0,2377	0,60
Sem Máscara de Proteção	349	0,5744	0,2164	0,48
Máscara de Proteção (EPI)	151	0,5614	0,1945	0,49
Contêiner de Canteiro	1.208	0,5641	0,2120	0,47
Extintor CO2 Babcock Davis	1.289	0,5539	0,2087	0,46
Empilhadeira	1.367	0,5228	0,1825	0,46
Capacete de Segurança (NR 6)*	30	0,4770	0,1455	0,45
Cone de Segurança	336	0,5153	0,1925	0,43
Extintor CO2 Walker MC-2A	781	0,3992	0,1278	0,40
Cadeira / Ambiente	1.900	0,4647	0,1745	0,40

Nota: A amostragem n contabiliza caixas delimitadoras individuais detectadas (total de 13.553 *bounding boxes*), diferindo do volume de requisições HTTP da Tabela 1. Para as classes Extintor CO2 Walker MC-2A e Cadeira/Ambiente, o valor calculado ficou abaixo do piso mínimo, sendo elevado para $\tau = 0,40$. As classes com amostragem reduzida (*), como Óculos ($n = 3$) e Capacete ($n = 30$), recebem essa marcação como ressalva sobre a confiabilidade estatística da média e desvio padrão, ainda que seus valores de τ não tenham sido limitados pelo piso.

A calibração dos limiares permitiu filtrar ruídos de fundo no processo real, elevando a precisão das inspeções operacionais e garantindo que detecções espúrias sejam descartadas antes da emissão do laudo.

CONCLUSÕES

O sistema de predição e análise contextual de cenas desenvolvido neste trabalho provou ser uma solução eficiente e de baixa latência para automação de inspeções. A utilização de redes neurais YOLO aceleradas por OpenVINO FP16 na GPU Intel Arc B570 proporcionou tempo de inferência médio de 12,06 ms por quadro no processo real com processamento 100% em memória RAM.

A introdução do motor de sobcamada e do modelo matemático de limiares dinâmicos (τ_c), respaldado pela análise telemetrada de 14.840 requisições reais de produção, possibilitou filtrar falsos alarmes

e atingir 79,86% de acertos no subconjunto multimodelo e 93,55% em modelos dedicados, enquanto a integração do LLM Ollama garantiu a geração autônoma de laudos técnicos explicativos. Trabalho futuro contempla a expansão do dataset de inspeção para novas categorias regulamentares.

AGRADECIMENTOS

Os autores agradecem ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP) Campus Campinas pelo suporte institucional e de infraestrutura disponibilizado para o desenvolvimento desta pesquisa.

REFERÊNCIAS

- BRASIL. *Norma Regulamentadora n. 18: Segurança e Saúde no Trabalho na Indústria da Construção*. Brasília, DF, 2020. Disponível em: <<https://www.gov.br/trabalho-e-emprego/pt-br/assuntos/inspecao-do-trabalho/seguranca-e-saude-no-trabalho/normas-regulamentadoras/nr-18.pdf>>. Acesso em: 17 jun. 2026.
- EVERINGHAM, M. et al. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, Springer, v. 88, n. 2, p. 303–338, 2010.
- FIELDING, R. T. *Architectural styles and the design of network-based software architectures*. Tese (Doutorado) — University of California, Irvine, Irvine, 2000.
- RAHMAN, M. A.; ULUSOY, I.; MEYDANLI, A. Context-aware object detection: A survey. *IEEE Access*, IEEE, v. 9, p. 123456–123470, 2021.
- REDMON, J. et al. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2016. p. 779–788.
- TERVEN, J.; CÓRDOVA-ESPARZA, D. A comprehensive review of yolo architectures in object detection. *Artificial Intelligence Review*, Springer, v. 56, n. 9, p. 8415–8474, 2023.
- ULTRALYTICS. *Ultralytics YOLO Documentation*. [S.l.], 2026. Disponível em: <<https://docs.ultralytics.com/>>. Acesso em: 17 jun. 2026.